

P4_1



P4_1

Question 1 (10 points)

Prove by induction that for

$$T(n) = \begin{cases} 2 & \text{if } n = 2, \\ 2 \cdot T\left(\frac{n}{2}\right) + n & \text{if } n = 2^k, \text{ for } k > 1 \end{cases} \quad (1)$$

$T(n) = n \lg n$, where $\lg n$ refers to $\log_2 n$.

If submitting a text file, use standard mathematical notation in writing your answer. For example, n^{2+1} would be written in the .txt file as $n^{(2+1)}$.

Base: $n = 2$

$$2 = 2 \lg 2$$

$$2 = 2 \log_2(2)$$

$$2 = 2(1) \quad \checkmark$$

induction hypothesis: $n = 2^k$

$$T(2^k) = 2^k \lg(2^k)$$

$$T(2^k) = k \cdot 2^k$$

$$\rightarrow T(2^{k+1}) = 2 \cdot T\left(\frac{2^{k+1}}{2}\right) + 2^{k+1} \quad \frac{2^{k+1}}{2} = 2^{k+1} \cdot 2^{-1} = 2^k$$

$$T(2^{k+1}) = 2 T(2^k) + 2^{k+1}$$

$$T(2^{k+1}) = 2k \cdot 2^k + 2^{k+1}$$

$$k 2^{k+1} + 2^{k+1}$$

$$T(2^{k+1}) = 2^{k+1} (k+1)$$

$$2^{k+1} \log_2(2^{k+1}) = 2^{k+1} (k+1)$$

$$2^{k+1} (k+1) = 2^{k+1} (k+1) \quad \checkmark$$

Question 2 (10 points)

Analyze the worst-case time complexity of the following pseudocode of a mystery algorithm. Show your work.

```
bool mystery(Tasks, Cores, targetTemp)
// Tasks and Cores are sets of sizes |Tasks| and |Cores|, respectively.
// targetTemp is a double.
{
    bool flag = false;
    while (Tasks != null) loop 1  $O(|Tasks|)$ 
    {
        for (each task in Tasks) loop 2  $O(1 \text{ task})$  rest of Tasks from outer loop
        {
            flag = false;
            for (each core in Cores) loop 3  $O(|Cores|)$ 
            {
                if (!core.isBusy()) //  $O(1)$ 
                {
                    peakTemp = solve(Cores); //  $O(|Cores|^2)$ 
                }

                if (peakTemp <= targetTemp)
                {
                    assign task to core; //  $O(1)$ 
                    Tasks <- Tasks - task // Remove task from Tasks,  $O(1)$ 
                    flag = true;
                    break;
                }
            }
        }

        if (!flag) // Can't assign this task
        {
            return flag;
        }
    }

    for (each core in Cores) loop 4  $O(|Cores|)$ 
    {
        //Perform housekeeping --  $O(1)$ 
    }
}

return true;
}
```

$$O(|Tasks|^2 \cdot |Cores|^3)$$

Question 3 (10 points)

Formally, $f(x)$ is $O(g(x))$ if there exist constants C and k such that

$$|f(x)| \leq C|g(x)|, \quad (2)$$

where $x > k$. Note that choices of C and k are not unique.

Given this,

(a) (5 points) is 2^{n+5} $O(2^n)$?

(b) (5 points) is 2^{3n} $O(2^n)$?

Show your work.

$$a) \quad 2^{n+5} = 2^n \cdot 2^5 = 2^n \cdot 32$$

set up in equality:

$$2^{n+5} \leq C \cdot 2^n$$

$$C = 32 \quad k = 0$$

$$x > k \quad 2^n \cdot 32 \leq 32 \cdot 2^n \quad \text{so } O(2^n) \text{ with } C = 32 \quad k = 0$$

$$b) \quad 2^{3n} = (2^n)^3 = 2^n \cdot 2^n \cdot 2^n$$

$$2^{3n} \leq C \cdot 2^n$$

$$2^{2n} \leq C$$

C need to increase exponentially like n
meaning C can't be a constant number
so it is not $O(2^n)$

Question 4 (10 points)

Given an array of integers a , design an algorithm that removes all the (non-zero) negative integers while preserving the relative order of the positive integers without using additional vectors. The remaining elements of the array a should then be filled with 0's. Your algorithm should have a time complexity of $O(n)$ where n is the total number of elements in a . Follow the following declaration, which returns the total number of positive integers in the array a with n elements.

```
int removeNegInt(int a[], int n)
```

```
    for (int i=0; i < n; i++) {  
        if (a[i] > 0) {  
            a[i] = 0; // set negative numbers to 0  
        }  
        else if (a[i] < 0) {  
            a[pos] = a[i] // put positive numbers at beginning  
            pos++; // increase number of positive values  
        }  
    }  
    return pos;  
}
```